JavaScript Operators

Javascript operators are used to perform different types of mathematical and logical computations.

Examples:

The **Assignment Operator =** assigns values

The **Addition Operator +** adds values

The **Multiplication Operator *** multiplies values

The **Comparison Operator >** compares values

JavaScript Assignment

The **Assignment Operator** (=) assigns a value to a variable:

Assignment Examples

```
let x = 10;

// Assign the value 5 to x
let x = 5;
// Assign the value 2 to y
let y = 2;
```

JavaScript Addition

The **Addition Operator** (+) adds numbers:

Adding

```
let x = 5;
let y = 2;
let z = x + y;
```

JavaScript Multiplication

The **Multiplication Operator** (*) multiplies numbers:

Multiplying

```
let x = 5;
let y = 2;
let z = x * y;
```

Types of JavaScript Operators

There are different types of JavaScript operators:

- Arithmetic Operators
- Assignment Operators
- Comparison Operators
- String Operators
- Logical Operators
- Bitwise Operators
- Ternary Operators
- Type Operators

JavaScript Arithmetic Operators

Arithmetic Operators are used to perform arithmetic on numbers:

Arithmetic Operators Example

```
let a = 3;
let x = (100 + 50) * a;
```

Operator	Description
+	Addition

-	Subtraction
*	Multiplication
**	Exponentiation (ES2016)
/	Division
%	Modulus (Division Remainder)
++	Increment
--	Decrement

Arithmetic Operations

A typical arithmetic operation operates on two numbers.

The two numbers can be literals:

Example

```
let x = 100 + 50;
```

Operators and Operands

The numbers (in an arithmetic operation) are called **operands**.

The operation (to be performed between the two operands) is defined by an **operator**.

Operand	Operator	Operand
100	+	50

Adding

The **addition** operator (`+`) adds numbers:

Subtracting

The **subtraction** operator (`-`) subtracts numbers.

Multiplying

The **multiplication** operator (`*`) multiplies numbers.

Example

```
let x = 5;
let y = 2;
let z = x * y;
```

Dividing

The **division** operator (`/`) divides numbers.

Example

```
let x = 5;
let y = 2;
let z = x / y;
```

Remainder

The **modulus** operator (`%`) returns the division remainder.

Example

```
let x = 5;
let y = 2;
let z = x % y;
```

In arithmetic, the division of two integers produces a **quotient** and a **remainder**.

In mathematics, the result of a **modulo operation** is the **remainder** of an arithmetic division.

Incrementing

The **increment** operator (`++`) increments numbers.

Example

```
let x = 5;
x++;
let z = x;
```

Decrementing

The **decrement** operator (`--`) decrements numbers.

Example

```
let x = 5;
x--;
let z = x;
```

Exponentiation

The **exponentiation** operator (`**`) raises the first operand to the power of the second operand.

Example

```
let x = 5;
let z = x ** 2;
```

x ** y produces the same result as `Math.pow(x,y)`:

JavaScript Assignment Operators

Assignment operators assign values to JavaScript variables.

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

The += Operator

The **Addition Assignment Operator** adds a value to a variable.

Addition Assignment Examples

```
let x = 10;
x += 5;
```

The -= Operator

The **Subtraction Assignment Operator** subtracts a value from a variable.

Subtraction Assignment Example

```
let x = 10;
x -= 5;
```

The *= Operator

The **Multiplication Assignment Operator** multiplies a variable.

Multiplication Assignment Example

```
let x = 10;
x *= 5;
```

The **= Operator

The **Exponentiation Assignment Operator** raises a variable to the power of the operand.

Exponentiation Assignment Example

```
let x = 10;
x **= 5;
```

The /= Operator

The **Division Assignment Operator** divides a variable.

Division Assignment Example

```
let x = 10;
x /= 5;
```

The %= Operator

The **Remainder Assignment Operator** assigns a remainder to a variable.

Remainder Assignment Example

```
let x = 10;
x %= 5;
```

JavaScript Objects

JavaScript Objects

You have already learned that JavaScript variables are containers for data values.

Objects are variables too. But objects can contain many values.

This code assigns **many values** (Fiat, 500, white) to a **variable** named car:

```
{type:"Fiat", model:"500 const car = ", color:"white"};
```

The values are written as **name:value** pairs (name and value separated by a colon).

It is a common practice to declare objects with the `const` keyword.

Object Definition

You define (and create) a JavaScript object with an object literal:

Example

```
const person = {firstName:"John", lastName:"Doe", age:50,
eyeColor:"blue"};
```

Spaces and line breaks are not important. An object definition can span multiple lines:

Example

```
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 50,
  eyeColor: "blue"
};
```

Object Properties

The **name:values** pairs in JavaScript objects are called **properties**:

Accessing Object Properties

You can access object properties in two ways:

objectName.propertyName

or

objectName["propertyName"]

Example1

```
person.lastName;
```

Example2

```
person["lastName"];
```

JavaScript objects are containers for **named values** called properties.